

AWS GenAI 課程 · 第 6 單元

EC2 與運算選型

盧建霖 • 2026-09-19

同一台 **m7g.large** ，
有人一小時付一塊，有人付一毛。
差別在你交出了多少彈性。

Agenda

- 01 讀懂型號：一個名字四段
- 02 定價模式：折扣用彈性換
- 03 Security Group：只有允許
- 04 開機與進機器：先烤好、不開 22
- 05 GPU 的帳：租車還是搭計程車
- 06 實作：第一台 EC2，不用金鑰進去

TODAY'S GOAL

替專題的三台機器定型
號、定付款、定門口，
再親手開一台不建金
鑰、不開 22 埠也進得去
的機器。

前兩段決定花多少錢，中間兩段決定
機器怎麼開、怎麼進，再算 GPU 的
帳；最後一段動手把它們接起來。

先定義名詞，後面才好講

這六個詞今天會反覆出現，第一次出現就先給名字

執行個體 **EC2 instance**

AWS 的虛擬機器服務叫 EC2，開出來的一台叫一個 instance。

競價 **Spot**

用 AWS 的閒置容量換折扣，隨時可能被收回，兩分鐘前通知。

預留執行個體 **RI**

Reserved Instances，承諾用一或三年換折扣。

安全群組 **Security Group**

掛在每台 EC2 上的虛擬防火牆，決定哪些連線能進能出。

映像檔 **AMI**

Amazon Machine Image，開機器用的映像，含作業系統跟磁碟內容。

無金鑰遠端維運 **Session Manager**

不開任何入站埠、不用金鑰，從主控台或 CLI 進機器的 shell。

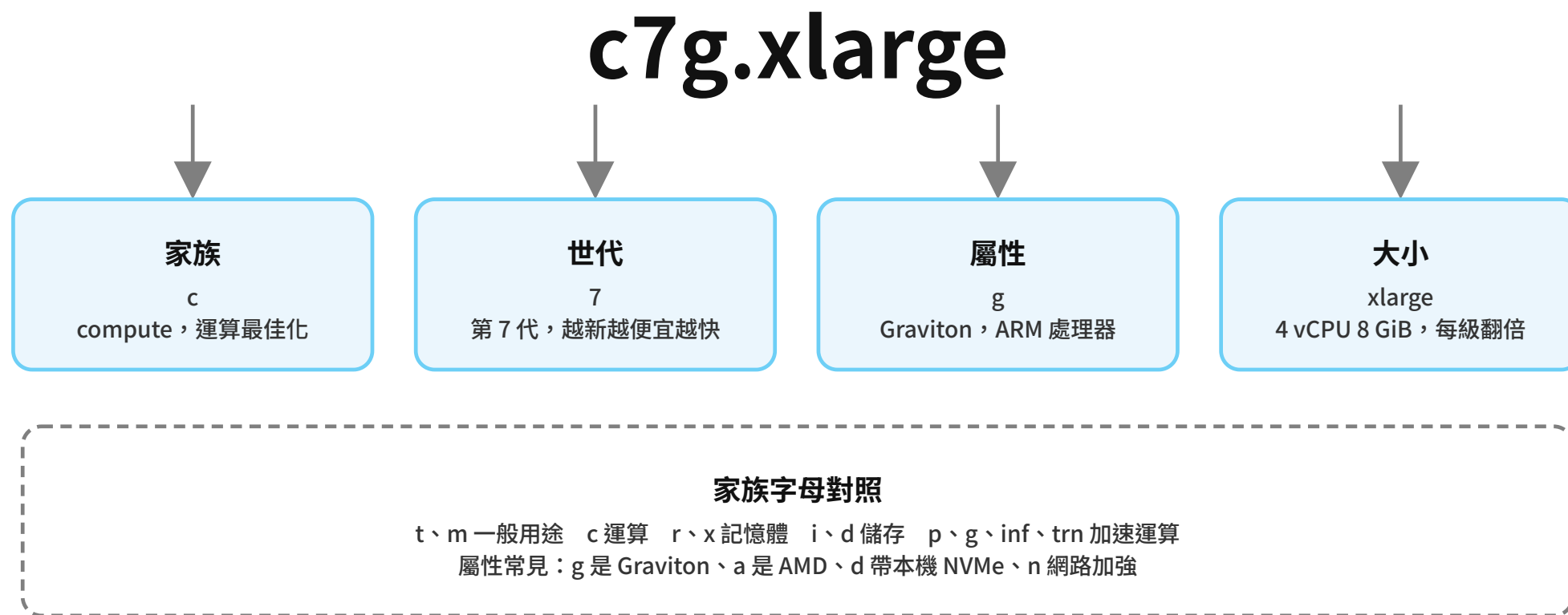
INSTANCE TYPES

型號不用背，會拆名字就夠

- 一、點前面三段講哪種機器，點後面一段講多大
- 二、家族字母決定每個 **vCPU** 配多少記憶體
- 三、先開小台，量測兩週再調

c7g.xlarge 拆成四段讀

四條箭頭把型號拆成家族、世代、屬性、大小



- 1 點前面三段講「這是哪種機器」，點後面一段講「多大」
- 2 g 是 Graviton：AWS 自家的 ARM 處理器，同效能便宜約兩成
- 3 底下虛線框是家族跟屬性的字母對照表，用查的不用背

每一段，選的時候看不同的東西

大小的錨點：large 通常是 2 vCPU，xlarge 是 4，2xlarge 是 8

段	例子	意思	選的時候看什麼
家族	c	這台機器偏重什麼資源	程式吃 CPU 還是吃記憶體
世代	7	第幾代硬體	同家族選最新，通常更快也更便宜
屬性	g	處理器或附加特性	g 是 Graviton，a 是 AMD
大小	xlarge	vCPU 與記憶體	每升一級大致翻倍，價格也翻倍

同家族內記憶體跟 vCPU 的比例固定：問完要多少記憶體，家族跟大小一起定。

家族字母，管的是記憶體比例

家族	字母	每 vCPU 記憶體	適合
一般用途	t、m	4 GiB	網站、API、開發機
運算最佳化	c	2 GiB	批次運算、影音轉檔、高效能運算
記憶體最佳化	r、x	8 GiB 以上	資料庫、記憶體內快取、大資料框
儲存最佳化	i、d	視型號	要本機 NVMe 高 IOPS 的資料庫
加速運算	p、g、inf、trn	視型號	訓練、推論、繪圖

吃記憶體的快取不適合 c，純算力工作不適合 r：選錯就是多付錢。

不知道選什麼，全部選 **m** 最保險？

× 大家通常會想

全部開 **m** 家族最保險，什麼都能跑。

一般用途每個 vCPU 配 4 GiB，CPU 跟記憶體都不缺。

✓ 實際上

可以，代價是吃 CPU 的工作多付了記憶體的錢。

先跑兩週，Compute Optimizer 看實際用量後，會建議該換哪個家族。

選型的三個實務判斷

t 家族是 burstable：平時給基準 CPU，累積點數後可以短時間爆衝

新專案優先選 Graviton

屬性帶 g，Python 跟 Node 幾乎不用改；例外是依賴只有 x86 版本的套件。

世代越新越好

第 7 代通常比第 5 代快又便宜；舊世代留給有舊軟體相容需求的人。

開發機用 t 家族最小台

低流量最省；持續高負載會被限速，那時候要換 m。

QUIZ 01 · CLF-C02

一家公司希望在多個 **Amazon EC2** 執行個體上運行 **CPU** 密集型工作負載。公司應該使用哪一種類型的 **EC2** 執行個體來滿足這些需求？

- A** 通用型執行個體 (General purpose instances)
- B** 計算優化型執行個體 (Compute optimized instances)
- C** 記憶體優化型執行個體 (Memory optimized instances)
- D** 儲存優化型執行個體 (Storage optimized instances)

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · CLF-C02 · ANSWER

一家公司希望多個 **Amazon EC2** 執行個體上運行 **CPU** 密集型工作負載。公司應該使用哪一種類型的 **EC2** 執行個體來滿足這些需求？

- ☐ A 通用型執行個體 (General purpose instances)
- ☒ B 計算優化型執行個體 (Compute optimized instances)
- ☐ C 記憶體優化型執行個體 (Memory optimized instances)
- ☐ D 儲存優化型執行個體 (Storage optimized instances)

B。「CPU 密集」直接對運算優化型，也就是 c 家族；記憶體密集才選 r。

動手做：c7g、m7g、r7g 並排，只換家族字母，記憶體就翻倍（1/2）

小步	做什麼	做完應該看到
1	主控台右上角 Region 切到 Asia Pacific (Tokyo)，進 EC2，左側導覽列點 Instance Types	一長串型號清單，有 vCPUs、Memory (GiB) 等欄位
2	清單上方的搜尋框輸入 7g.xlarge	清單縮到剩幾列，裡面有 c7g.xlarge、m7g.xlarge、r7g.xlarge
3	勾選 c7g.xlarge、m7g.xlarge、r7g.xlarge 三列	畫面下方出現並排比較：三台 vCPU 都是 4，記憶體分別是 8、16、32 GiB

動手做：c7g、m7g、r7g 並排，只換家族字母，記憶體就翻倍（2/2）

小步	做什麼	做完應該看到
4	點清單右上角的齒輪圖示，勾 On-Demand Linux pricing，按 Confirm	清單多一欄每小時價格；三台價格不同，記憶體越多的越貴
5	選做：點主控台上方導覽列的 CloudShell 圖示，貼下面那條指令	一張三列的表：型號、4、記憶體 MiB 數（8192、16384、32768）

世代、屬性、大小都一樣，只換家族字母，vCPU 不動、記憶體從每 vCPU 2 GiB 變 4 再變 8，這就是本課「同家族內記憶體跟 vCPU 的比例固定」那段。

每一分折扣，都是交出去的彈性

- 一、**Spot** 用「可以被中斷」換折扣
- 二、**RI** 跟 **Savings Plans** 用「承諾一年以上」換
- 三、第一個月先 **On-Demand**，量測完才承諾

四種模式加一個特例，折扣用彈性換

模式	白話	折扣	代價
On-Demand 隨需	用一小時付一小時，隨時開關	無	最貴
Reserved Instances	承諾一或三年，綁家族與 Region	三到七成	中途不用也要付
Spot 競價	用閒置容量，兩分鐘前通知收回	最多九成	不能保證跑完
Savings Plans 節省方案	承諾每小時花多少錢，不綁型號	跟 RI 差不多	一樣要承諾一或三年
Dedicated Hosts 專用主機	整台實體機，看得到核心跟插槽	沒有，最貴	只為帶舊軟體授權上雲

三個名字很像，解決的問題不同

題目常把它們放在同一組選項裡

Dedicated Instances

專用執行個體：只保證硬體不跟別的帳號共用。

Dedicated Hosts

專用主機：連實體核心的位置都固定；授權綁核心數的軟體要它。

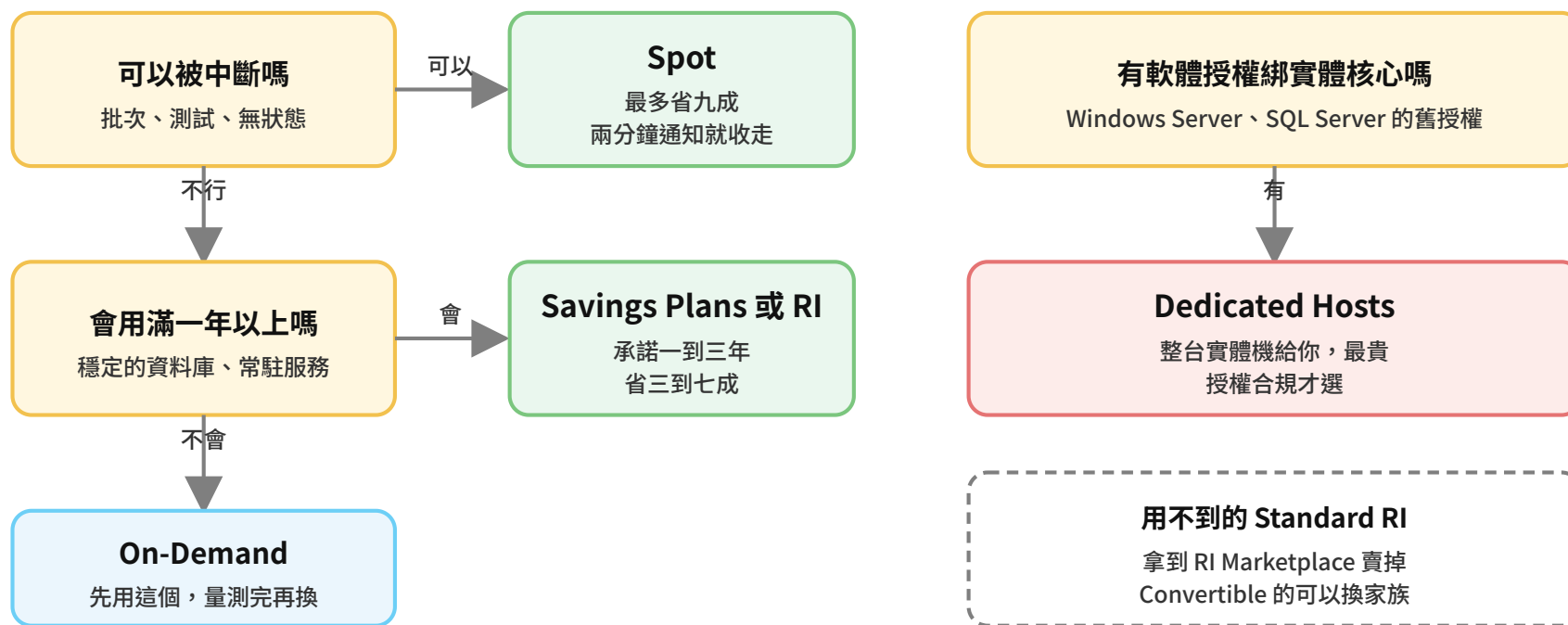
Capacity Reservations

容量預留：保證那一區有機器給你，但不打折；可以跟 RI 疊。

三個問題，由上往下問

左邊黃框是問題，「是」往右走到綠框的模式

先問三個問題，答案決定定價模式



1 第一問能不能被中斷：可以就 Spot

2 第二問會不會用滿一年：會就 Savings Plans 或 RI

3 第三問授權有沒有綁核心；三個都不是，On-Demand 先用著

RI 跟 Savings Plans，差在綁不綁型號

比較	Reserved Instances	Savings Plans
承諾什麼	家族、Region、一或三年	每小時花多少錢，一或三年
從 m6 換到 m7g	Standard 要重買，Convertible 可換	照算
用不到的時候	可以在 RI Marketplace 轉賣	不能轉賣

2026 年的預設是 Savings Plans；兩個都不要在第一個月買，先看 Cost Explorer 的穩定用量。

題目一定給一個「彈性」的線索

看到這些字	答案
可中斷、可容忍停機、批次、無狀態	Spot
一年、全年無休、已正確規模化	Standard RI (或 Savings Plans)
想換家族、想保留彈性	Convertible RI 或 Compute Savings Plans
每插槽、每核心、每 VM 授權	Dedicated Hosts
資料要留自己機房	不是定價題，是 Outposts

買了 RI 用不到：RI Marketplace 賣掉。找到線索就找到模式。

QUIZ 02 · CLF-C02

對於一個專案期為 **1 年**、需要全年無休運作且已正確規模化 (**right-sized**) 的資料庫伺服器而言，哪一種 **Amazon EC2** 計價模式可以提供「最多」的成本節省？

A On-Demand Instances

B Convertible Reserved Instances

C Spot Instances

D Standard Reserved Instances

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · CLF-C02 · ANSWER

對於一個專案期為 **1 年**、需要全年無休運作且已正確規模化 (**right-sized**) 的資料庫伺服器而言，哪一種 **Amazon EC2** 計價模式可以提供「最多」的成本節省？

A On-Demand Instances

B Convertible Reserved Instances

C Spot Instances

D Standard Reserved Instances

D。「一年、全年無休、已正確規模化」代表不會換機型，**Standard RI** 折扣最深；**Convertible** 多付錢買一個用不到的彈性，**Spot** 會被收回，資料庫不能接受。

動手做：查 m7g.large 的 Spot 價，算它比 On-Demand 便宜幾成

小步	做什麼	做完應該看到
1	Region 切到 Asia Pacific (Tokyo)，EC2 → 左側 Instance Types，搜尋 m7g.large；點右上齒輪勾 On-Demand Linux pricing → Confirm	m7g.large 那列多一欄每小時價格，記下來
2	左側導覽列點 Spot Requests，再點頁面上的 Pricing history	跳出價格歷史圖表視窗
3	Graph 選 Availability Zones；Instance type 選 m7g.large；Platform 選 Linux/UNIX；Date range 選最長的那個	圖上出現幾條線，一個可用區一條，線是慢慢起伏的
4	滑鼠移到圖上任一點	圖上方資訊區出現兩列價格：第一列是那天的價，第二列是整個區間的平均價，記下平均價
5	算：平均 Spot 價 ÷ 第 1 步的 On-Demand 價	一個百分比；一減掉它就是 Spot 的折扣

這個折扣換來的代價是「AWS 要回容量時會收走你的機器」，也就是本課決策圖第一問「能不能被中斷」在價格上的樣子。

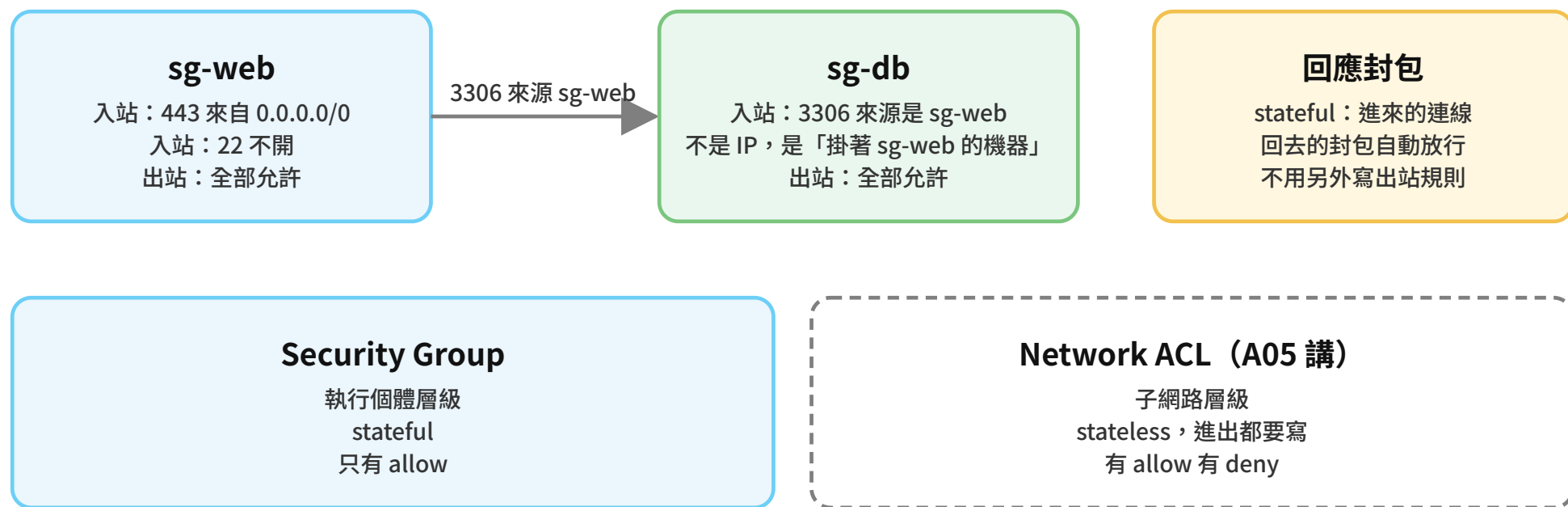
SG 只會說可以，不會說不行

- 一、掛在執行個體上、記得連線、只有允許
- 二、來源填另一個 **SG**：描述角色，不描述位址
- 三、要拒絕某個 **IP**，才去找 **Network ACL**

資料庫的來源，填的是另一個 SG

上排是兩個 SG 跟一條規則，下排是 Security Group 跟 Network ACL 的對照

Security Group 掛在執行個體上，規則只寫「允許」，來源可以是另一個 SG



- 1 **sg-db 的規則寫「3306 來源是 sg-web」：**掛著 sg-web 的任何機器
- 2 右邊黃框是 stateful（有狀態）：進來的連線，回去的封包自動放行
- 3 **sg-web 對全世界只開 443，22 不開**

三個性質，各帶一個後果

性質	意思	後果
執行個體層級	一台可掛多個，一個 SG 可掛很多台	改一個 SG，掛它的機器全部生效
stateful 有狀態	允許 443 進來，回應不用再寫出站	出站預設全開，通常只寫入站
只有 allow	沒寫就是拒絕，寫了就是允許	擋特定 IP 得用 Network ACL 或 WAF

新建的 SG 入站什麼都不允許、出站全部允許：[連不上](#)，先看有沒有開埠。

資料庫的來源，寫一段 IP 就好？

× 大家通常會想

資料庫的規則寫「來源 10.0.1.0/24」就好。

Auto Scaling 一擴，新機器的 IP 可能不在那一段。

✓ 實際上

來源填 sg-web 的 ID：掛著那個 SG 的任何機器。

機器換 IP、加台數都不用改規則；規則描述的是角色，不描述位址。

三條實務規則：**22** 永遠不對全世界開

規則的來源可以填 IP 範圍 (CIDR)、另一個 SG，或 prefix list

1 **0.0.0.0/0 只給 80 跟 443**

22 不對全世界開；今天稍後會讓你連 22 都不用開。

2 **一個角色一個 SG**

網站一個、資料庫一個、跳板一個，不要全部塞在 default 那個。

3 **SG 是免費的**

多開幾個沒成本。

兩道門：NACL 在子網路，SG 在機器

Network ACL（網路存取控制清單）：掛在子網路上的無狀態防火牆；封包兩道都過才進得來

面向	Security Group	Network ACL
掛在哪	執行個體	子網路
狀態	stateful	stateless，回應封包也要寫規則
規則	只有 allow	allow 跟 deny 都有，依編號比對
用途	日常主力	擋特定 IP、子網路級的粗篩

擋 SQL injection 兩個都不行，那是 WAF（看 HTTP 內容的防火牆）的事。

QUIZ 03 · CLF-C02

公司可以透過在 **AWS Cloud** 中使用 **security groups** 來執行下列哪一項工作？

- ☐ A 僅允許透過特定連接埠 (port) 存取 Amazon EC2 執行個體。
- ☐ B 在子網路 (subnet) 層級拒絕來自惡意 IP 位址的存取。
- ☐ C 保護由 Amazon CloudFront 快取的資料。
- ☐ D 將無狀態 (stateless) 防火牆套用至 Amazon EC2 執行個體。

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · CLF-C02 · ANSWER

公司可以透過在 **AWS Cloud** 中使用 **security groups** 來執行下列哪一項工作？

- A** 僅允許透過特定連接埠 (port) 存取 Amazon EC2 執行個體。
- B 在子網路 (subnet) 層級拒絕來自惡意 IP 位址的存取。
- C 保護由 Amazon CloudFront 快取的資料。
- D 將無狀態 (stateless) 防火牆套用至 Amazon EC2 執行個體。

A。Security Group 只能寫允許，規則由協定、埠、入站來源或出站目的地組成；B「在子網路層級拒絕」是 Network ACL 的事，D stateless 也是 NACL，SG 是 stateful。

動手做：打開 **default SG** 跟 **default NACL**，兩張規則表並排比 (1/2)

小步	做什麼	做完應該看到
1	Region 切到 Asia Pacific (Tokyo)，EC2 → 左側 Security Groups	列表至少一列，Security group name 是 default
2	點 default 那列的 Security group ID，記下它的 VPC ID，切到 Inbound rules 分頁	沒人改過的話是一條規則：Protocol 跟 Port range 都是 All，Source 是 sg- 開頭的 ID，而且就是這個 SG 自己的 ID
3	切到 Outbound rules 分頁	沒人改過的話是 All traffic，Destination 是 0.0.0.0/0 (VPC 有開 IPv6 會再多一條 ::/0)：出站預設全開

動手做：打開 **default SG** 跟 **default NACL**，兩張規則表並排比 (2/2)

小步	做什麼	做完應該看到
4	回頭看入站跟出站兩張表的欄位名	沒有任何一欄叫 Allow 或 Deny：寫了就是允許
5	上方搜尋列輸入 VPC 進 VPC 主控台 → 左側 Network ACLs → 找 VPC ID 跟第 2 步相同、Default 是 Yes 的那一列，點它的 ID → Inbound rules 分頁	沒人改過的話 Rule number 有 100 跟 *，最後一欄 Allow/Deny：100 是 ALLOW，* 是 DENY（有 IPv6 會多一條 101）

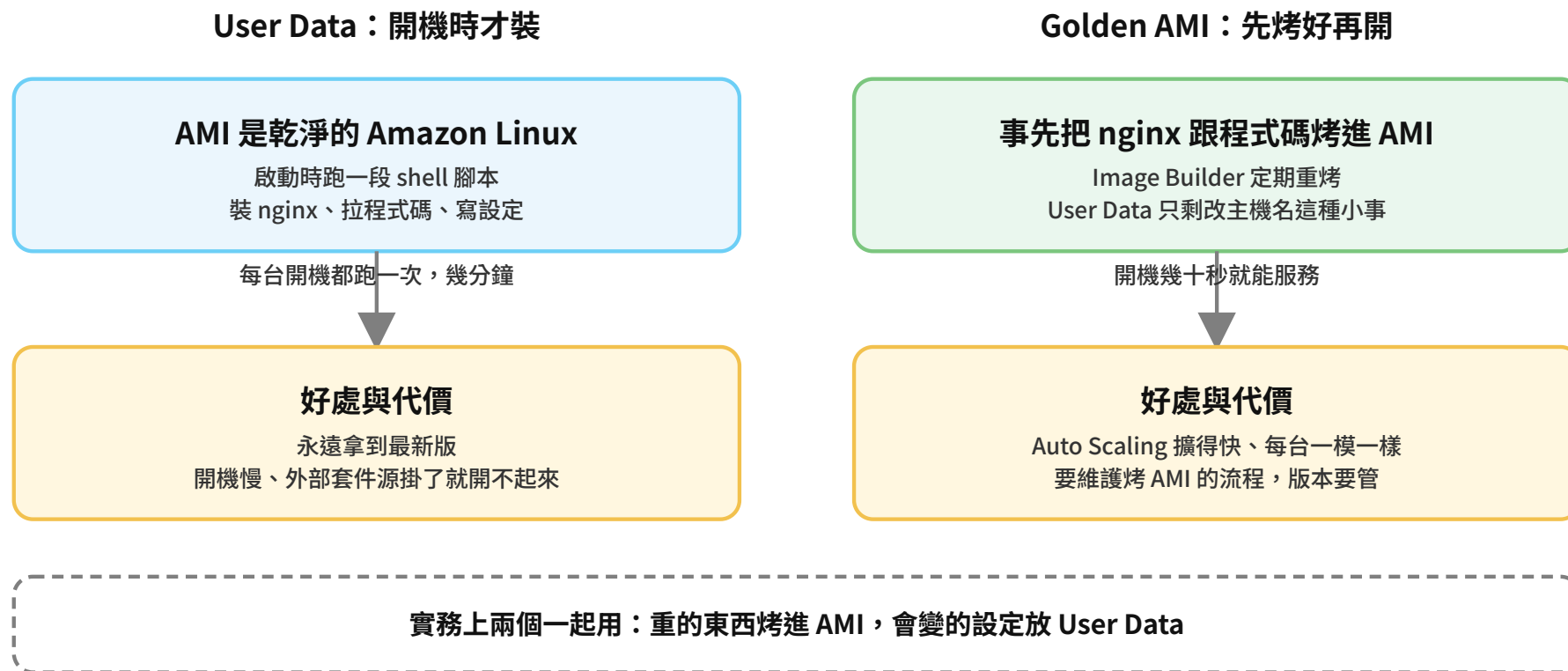
第 2 步的來源填的是一個 SG，意思是「掛這個 SG 的機器彼此全通」，就是本課「來源可以是另一個 SG」；第 5 步有編號、有 DENY，就是 NACL 跟 SG 的差別。

先烤好再開，不開 22 也進得去

- 一、重的烤進 **AMI**，會變的放 **User Data**
- 二、**User Data** 只跑第一次，失敗不擋開機
- 三、**Session Manager**：機器自己連出去，**IAM** 決定誰能進

開機時裝，還是先烤好再開

User Data：開機時自動執行的腳本；Golden AMI：把系統、軟體、設定都烤進去的自訂映像



- 1 左邊藍框：乾淨的 AMI 開機時跑腳本，每台都要等幾分鐘
- 2 右邊綠框：先烤好再開，幾十秒就能服務，每台一模一樣
- 3 底下虛線框是搭法：重的烤進 AMI，會變的設定放 User Data

擴一台要幾分鐘還是幾十秒

面向	User Data	Golden AMI
開機到能服務	幾分鐘，看腳本裝多少東西	幾十秒
版本一致性	每台抓當下最新套件，可能不一樣	每台一模一樣
外部依賴	套件源掛了就開不起來	不依賴外部
維護成本	改腳本就好	要有重烤的流程
適合	開發機、單台、常改的東西	Auto Scaling、正式環境

User Data 長這樣：第一行不能少

一段 bash，開機時自動執行

```
user data

#!/bin/bash
dnf install -y nginx
systemctl enable --now nginx
echo "hello from this instance" > /usr/share/nginx/html/index.html
```

- 1 第 1 行 shebang 不能少，腳本以 root 跑
- 2 只在第一次啟動跑一次，重開機不再跑
- 3 log 在 /var/log/cloud-init-output.log

User Data 的三個坑

腳本寫得出來，不代表機器開得起來

失敗不擋開機

腳本失敗，機器會「開好但沒服務」；去看 cloud-init 的 log。

不要把秘密寫進去

任何能讀機器 metadata 的程式，都看得到 User Data。

長腳本要拆

拆成「烤進 AMI」跟「開機時改」，不然每台開機都在等 dnf。

烤 AMI 是一條有版本的流水線

EC2 Image Builder：AWS 的映像烤製服務，定義配方、排程重烤、自動測試

1 基底 AMI 進 Image Builder

裝軟體、跑測試；每週或基底有更新時重烤。

2 產出打了版本標籤的 Golden AMI

例如 Golden AMI v12。

3 Launch Template 指向它

Auto Scaling 用它開機。

跟 Docker image 同一個思想：環境是有版本的產物；差別是 AMI 包含整個作業系統。

QUIZ 04 · CLF-C02

一家公司希望能自動新增與移除 **Amazon EC2 執行個體 (instances)**，並且讓 **EC2 執行個體** 能夠動態因應不同的工作負載。哪一項服務或功能可以滿足這些需求？

A

Amazon DynamoDB

B

Amazon EC2 Spot Instances

C

AWS Snow Family

D

Amazon EC2 Auto Scaling

想 30 秒，等一下翻下一頁對答案。

QUIZ 04 · CLF-C02 · ANSWER

一家公司希望能自動新增與移除 **Amazon EC2 執行個體 (instances)**，並且讓 **EC2 執行個體** 能夠動態因應不同的工作負載。哪一項服務或功能可以滿足這些需求？

A Amazon DynamoDB

B Amazon EC2 Spot Instances

C AWS Snow Family

D Amazon EC2 Auto Scaling

D。「自動增減 EC2、動態因應負載」是 Auto Scaling；它靠的正是 Launch Template 跟 Golden AMI，機器才能在幾十秒內開好。

動手做：在開機精靈裡找到「先烤好」跟「開機再裝」那兩格 (1/2)

小步	做什麼	做完應該看到
1	Region 切到 Asia Pacific (Tokyo) ， EC2 → Launch instance	開機精靈頁面，右邊有 Summary 欄
2	Application and OS Images (Amazon Machine Image) 區塊，Quick Start 點 Amazon Linux	下方出現 AMI 名稱跟一串 ami- 開頭的 ID
3	點 Browse more AMIs	四個分類：Quickstart AMIs 、 My AMIs 、 AWS Marketplace AMIs 、 Community AMIs

動手做：在開機精靈裡找到「先烤好」跟「開機再裝」那兩格 (2/2)

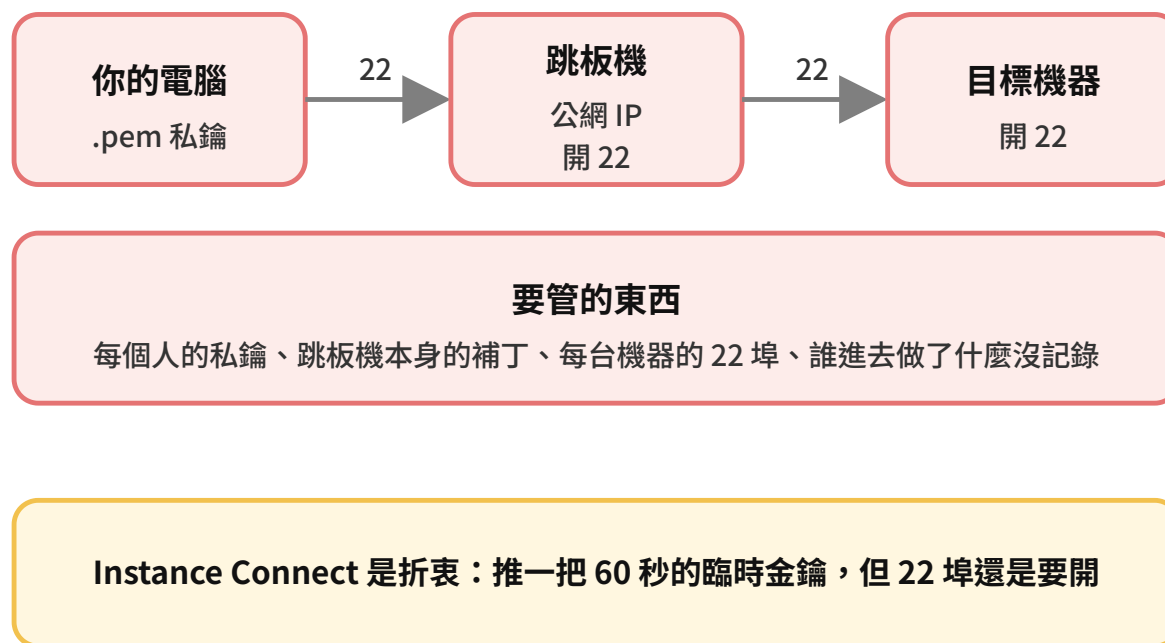
小步	做什麼	做完應該看到
4	點 My AMIs，選 Owned by me，看完按右上角 Cancel 回精靈	通常是空的：這個 Region 你名下沒有自己的 AMI；以後烤好的 Golden AMI 就出現在這
5	捲到最下面展開 Advanced details，找到 User data 欄位，貼本課那段四行腳本	腳本出現在框裡；框下方的 User data has already been base64 encoded 不要勾
6	清理：不按 Launch instance，直接點左側導覽列 Instances 離開精靈	Instances 列表沒有多出任何機器

同一頁表單上，AMI 那格決定「先烤好帶進來的」，User data 那格決定「開機後才跑的」，本課兩條路就是這兩格。

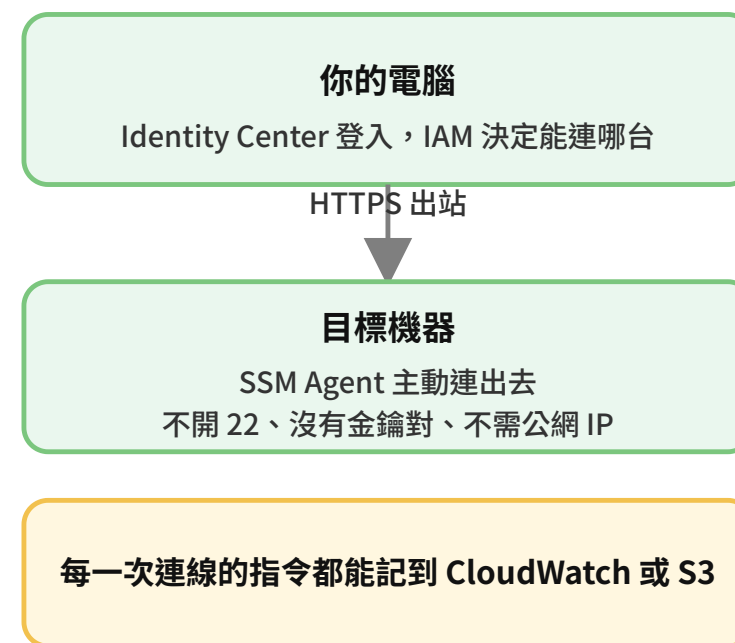
機器自己走出去，把你接進來

Session Manager：Systems Manager 的功能，不開入站埠、不用金鑰，就能進機器的 shell

舊做法：SSH 加跳板機



Session Manager



- 1 左邊紅色是舊做法：私鑰經跳板機的 22，再到目標機器的 22
- 2 右邊綠色：機器上的 SSM Agent 主動連出去，你從 IAM 進來
- 3 底下黃框：指令能記到 CloudWatch 或 S3；Instance Connect 是折衷

SSH 的問題：逼你管四樣東西

跳板機 (bastion host)：放在公網、唯一開 22 的那台，其他機器只信它

要管的	出事的樣子
私鑰 .pem	離職的人還留著；筆電丟了；貼進 repo
跳板機	它自己要補丁、要監控、要多花一台的錢
22 埠	每台機器多一個對外面
稽核	誰在幾點跑了什麼指令，SSH 不留

私鑰就是長期憑證：要靠人去刪；短期憑證到期自己作廢。

四個前提，沒有一個是開埠或金鑰

前提	做法
機器上有 SSM Agent	Amazon Linux 跟多數官方 AMI 預裝
機器的 Role 有 AmazonSSMManagedInstanceCore	掛一份 AWS 管理的 policy
機器能連出去到 SSM 端點	有公網出口，或用 VPC endpoint
你的身分有 ssm:StartSession	Resource 寫到 instance ARN，可以只准連某幾台

機器完全沒有入站：攻擊面從「一個對外的 22」變成零。

預設 **Session Manager**，跳板機是歷史

EC2 Instance Connect：AWS 幫你推一把 60 秒有效的臨時公鑰進機器

連法	開不開 22	金鑰	稽核	什麼時候用
傳統 SSH 加跳板	開	長期 .pem	無	舊環境還沒搬
EC2 Instance Connect	開，來源可限 AWS 的 IP 段	臨時	有 API 記錄	真的需要 SSH 協定
Session Manager	不開	無	每個指令可記錄	預設答案

QUIZ 05 · CLF-C02

一家公司使用最新的 **Amazon Linux 2 Amazon Machine Image (AMI)** 啟動了一個 **Amazon EC2** 執行個體。系統管理員可以採取哪些動作來連線至該 **EC2** 執行個體？（選兩項）

- ☐ A 使用 Amazon EC2 Instance Connect。
- ☐ B 使用遠端桌面通訊協定 (Remote Desktop Protocol, RDP) 連線。
- ☐ C 使用 AWS Batch。
- ☐ D 使用 AWS Systems Manager Session Manager。
- ☐ E 使用 Amazon Connect。

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · CLF-C02 · ANSWER

一家公司使用最新的 **Amazon Linux 2 Amazon Machine Image (AMI)** 啟動了一個 **Amazon EC2** 執行個體。系統管理員可以採取哪些動作來連線至該 **EC2** 執行個體？（選兩項）

- ☒ A 使用 Amazon EC2 Instance Connect。
- ☐ B 使用遠端桌面通訊協定 (Remote Desktop Protocol, RDP) 連線。
- ☐ C 使用 AWS Batch。
- ☒ D 使用 AWS Systems Manager Session Manager。
- ☐ E 使用 Amazon Connect。

A、D。Amazon Linux 2 的機器可以用 EC2 Instance Connect 或 Session Manager 連進去，兩個都不用自己管金鑰；RDP 是 Windows 的，Batch 是批次運算，Amazon Connect 是客服中心。

動手做：在主控台找到 **Session Manager** 的前提跟日誌開關 (1/2)

小步	做什麼	做完應該看到
1	IAM → 左側 Policies，搜尋 AmazonSSMManagedInstanceCore，點進去，權限內容切到 JSON	動作清單裡有 <code>ssmmessages:CreateControlChannel</code> 、 <code>ssmmessages:OpenDataChannel</code> 、 <code>ssm:UpdateInstanceInformation</code> ；整份沒有任何跟埠有關的東西
2	Region 切到 Asia Pacific (Tokyo)，搜尋列輸入 Systems Manager 進去，左側導覽列點 Session Manager	頁面有 Start session 按鈕，下面有 Session history、Preferences 等分頁
3	按 Start session	Target instances 清單：新帳號通常是空的，因為沒有滿足前提的機器；有東西也只看不選，看完離開這一頁

動手做：在主控台找到 Session Manager 的前提跟日誌開關 (2/2)

小步	做什麼	做完應該看到
4	回 Session Manager，點 Session history 分頁	新帳號通常是空的；有紀錄就看一下欄位，這頁只列最近的工作階段
5	點 Preferences 分頁，只看不按 Edit	有 CloudWatch logging 跟 S3 logging 的設定：一般 shell 工作階段裡跑的指令，要在這裡指定存到哪才會留下紀錄

第 1 步那幾個 `ssmmessages` 動作是機器上的 `agent` 往外連用的，這份 `policy` 只負責機器那一端的權限，所以不用開任何入站埠；`agent`、連得出去、你自己的權限另外三個前提還是要齊，第 3 步清單空的就是前提沒齊的樣子。

使用率不到六成，自架 **GPU** 就輸

- 一、**p** 是訓練用的大砲，**g** 是推論用的步槍
- 二、自架的錢花在「開著」，**token** 的錢花在「用到」
- 三、批次工作進度落地，就能用 **Spot** 中斷重跑

p 訓練、g 推論：月費從七百到數萬美元

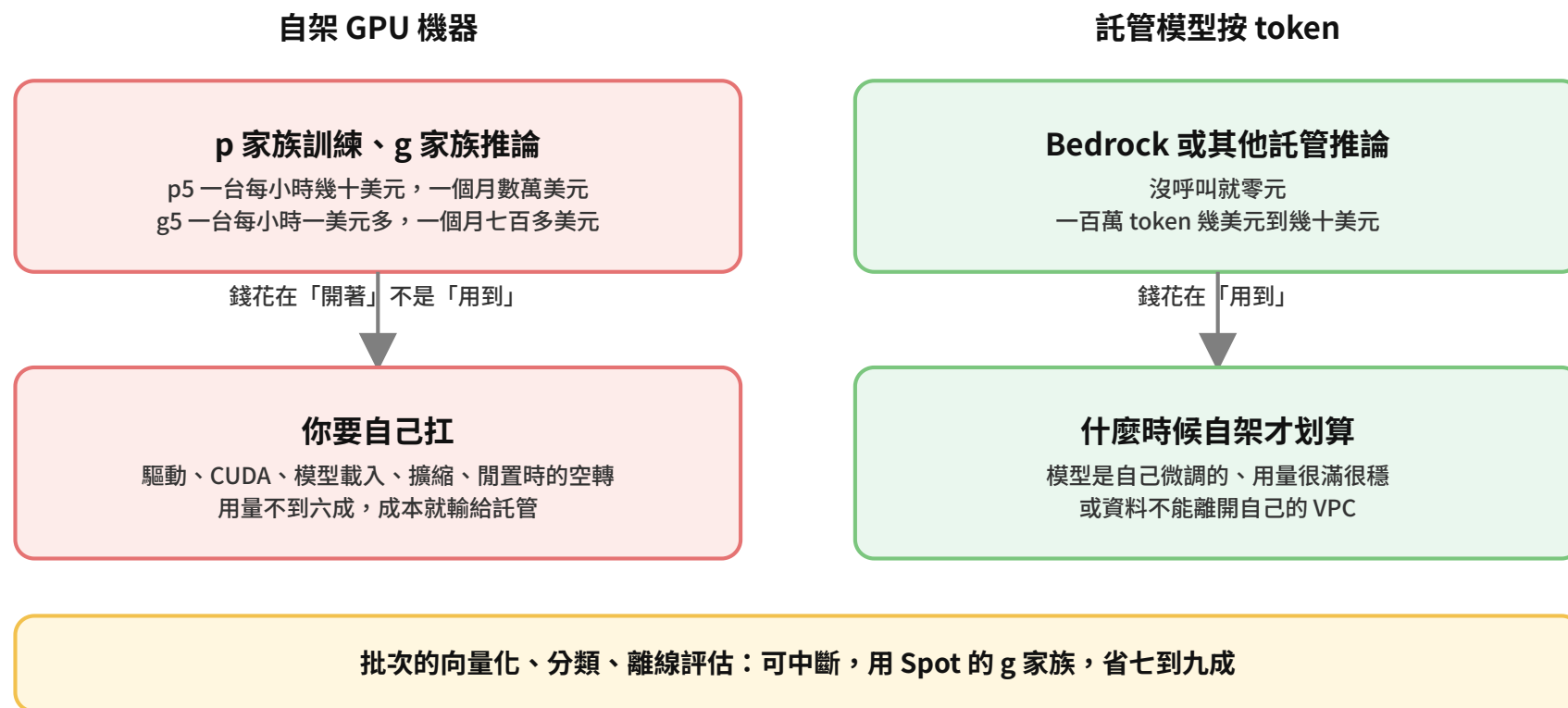
美東 On-Demand 價格；p5.48xlarge 是 p 家族第 5 代最大台，八張 H100

家族	定位	晶片例子	On-Demand 大約月費
p	訓練、大模型推論	p4 的 A100、p5 的 H100	p5.48xlarge 一個月數萬美元
g	推論、繪圖、小模型微調	g5 的 A10G、g6 的 L4	g5.xlarge 一個月七百多美元
inf	推論，AWS 自家 Inferentia	inf2	比同級 g 便宜，但模型要編譯過
trn	訓練，AWS 自家 Trainium	trn1	同上

價格每季在變，數字去 EC2 On-Demand Pricing [查當天的](#)。

自架的錢花在「開著」，託管花在「用到」

Bedrock 這類託管模型服務按 token 計費：沒呼叫就零元



- 1 左邊紅色：GPU 機器開著就算錢，還要自己扛驅動跟擴縮
- 2 右邊綠色：按 token 付；底下那格是自架才划算的條件
- 3 最底下黃框：批次工作可中斷，用 Spot 的 g 家族

用量不滿，自架就輸

面向	自架 g5.xlarge	Bedrock 按 token
沒人用的時候	一樣付，一個月七百多美元	零
尖峰	頂到就要開第二台，冷啟動幾分鐘	AWS 那邊擴
你要管	驅動、CUDA、模型載入、監控、補丁	什麼都不用
模型	自己下載、自己更新	選單裡選
什麼時候贏	用量很滿很穩、自己微調、資料不能離開 VPC	其他全部

Spot 跑批次：進度要落地

向量化 (embedding)：把文字變成一串數字；它跟分類都能中斷重跑，g 家族的 Spot 省七到九成

1 待處理的文件放在 S3

Spot 的 g5 或 inf2 拿來做，可被中斷。

2 每批做完就寫回

結果寫回 S3，做到哪記在 DynamoDB。

3 被收回就換新的 Spot 機器

從上次進度接著跑，不從頭。

專題第一次向量化先用 Bedrock 的 embedding 模型：幾萬份文件也就幾美元；百萬級、每天重跑才開 Spot GPU。

“

GPU 機器是租一整台車，token 是搭計程車；
你一天坐兩趟就別買車。

—— 一句話帶走 ——

QUIZ 06

專題每天大約叫模型三百次，每次幾百個 **token**，該自架 **g5** 還是用 **Bedrock** ？

A

自架，有上限比較安心

B

Bedrock，用量不到一成，付上限等於付十倍

C

自架 p5，反正未來會用到

D

兩個都開

想 30 秒，等一下翻下一頁對答案。

QUIZ 06 · ANSWER

專題每天大約叫模型三百次，每次幾百個 **token**，該自架 **g5** 還是用 **Bedrock** ？

A 自架，有上限比較安心

B Bedrock，用量不到一成，付上限等於付十倍

C 自架 p5，反正未來會用到

D 兩個都開

B。三百次乘幾百 **token**，一天不到一美元；**g5** 一個月七百多美元，開著不管有沒有人叫。

動手做：算一台 **GPU** 機器開一個月多少錢，再看你的帳號開不開得起來 (1/2)

小步	做什麼	做完應該看到
1	Region 切到 Asia Pacific (Tokyo)，EC2 → 左側 Instance Types；點右上齒輪勾 On-Demand Linux pricing → Confirm	清單多一欄每小時價格
2	搜尋 g5.xlarge，點型號名稱進詳細頁	規格裡寫一張 NVIDIA A10G GPU、4 vCPU、16 GiB 記憶體
3	回清單記下 g5.xlarge 每小時價格，乘 730（一個月大約 730 小時）	一個月的金額，跟本課表格的「七百多美元」同一個量級

動手做：算一台 GPU 機器開一個月多少錢，再看你的帳號開不開得起來 (2/2)

小步	做什麼	做完應該看到
4	同樣搜尋 p5.48xlarge，看規格再乘 730	八張 NVIDIA H100、192 vCPU；一個月的金額是 g5 的幾十倍
5	打開 Service Quotas 的 EC2 配額頁，篩選框輸入 On-Demand，找 Running On-Demand G and VT instances	Applied quota value 欄一個 vCPU 數字；小於 4 就代表這個帳號現在連一台 g5.xlarge 都開不起來

第 3、4 步把本課「開著就算錢」變成一個月的數字；第 5 步是另一道門，G 家族的配額官方預設是 0 vCPU，要自架得先申請，這也是專題先用 Bedrock 的理由之一。

四樣東西，親手接成一台機器

- 一、**Role**、**SG**、**User Data**、**Session Manager** 今天各做一次
- 二、瀏覽器走 **80**，你走 **Session Manager**，都不經過 **22**
- 三、機器活著不代表服務活著，**EC2** 的檢查看不出來

前面的動手做，在這裡接起來

動手做	在 Lab 哪一步再用到
c7g、m7g、r7g 並排比記憶體	第 3 步選機型 t3.micro
查 m7g.large 的 Spot 價	加分：查 Spot 價格，判斷這台適不適合 Spot
default SG 跟 default NACL 並排比	第 2 步建 web-sg，入站只開 HTTP
開機精靈裡的 AMI 跟 User data 兩格	第 3 步選 Amazon Linux 2023、第 4 步貼 User data
Session Manager 的前提跟日誌開關	第 1 步建掛 AmazonSSMManagedInstanceCore 的 Role、第 6 步用 Session Manager 連進去
GPU 機器一個月多少錢	加分：每小時價格乘一個月，算忘了關要付多少

一句話：前面六個動手做都只看不開，這裡第一次把看過的每一格真的填下去、開一台機器。

不建金鑰、不開 22，照樣進得去

上排五格是必做的順序，下排虛線框是進機器之後多玩的那一步

第一台 EC2：四樣東西接起來，不用金鑰也不開 22



多玩一步：停掉 nginx，網頁掛了，EC2 狀態檢查還是 2/2 通過（機器活著、服務死了）

開始前：用有管理者權限的身分登入主控台，右上角區域切到 Asia Pacific (Tokyo)

- 1** 上排照順序做：Role、SG、開機器、看網頁、進機器
- 2** 最後一格上色：不用金鑰就進得去，是這次實作的重點
- 3** 下排虛線框：停掉 nginx，網頁掛了，狀態檢查還是 2/2

三層：只交必做就及格

零程式碼，User Data 那段直接複製；評分是必做 70%、進階 20%、加分 10%

必做：七步

照做一定過：建 **Role**、建 **SG**、開機器、填進階設定、看網頁、進機器、清理。

進階：存成模板

清理之前，把這台機器的設定存成 **Launch Template**。

加分：算一筆帳

查 **t3.micro** 的時價，算忘了關一個月付多少，再判斷適不適合 **Spot**。

實作：第 1、2 步，Role 給機器掛、SG 只開 80

Role 讓機器能被 Session Manager 管；瀏覽器從 80 進來看網頁，22 不開

- 1 建 Role：IAM → Roles → Create role，Trusted entity 選 AWS service、Use case 選 EC2
- 2 勾 AmazonSSMManagedInstanceCore，名稱填 ec2-ssm-role
做完應該看到：Roles 列表出現 ec2-ssm-role
- 3 建 SG：EC2 → Security Groups → Create，名稱填 web-sg
- 4 Inbound 只加一條 HTTP、Source 選 My IP；Outbound 不動
做完應該看到：入站只有一條 80，沒有 22

實作：第 3 步，開機器不選金鑰

EC2 → Launch instance，由上往下填這五格

小步	做什麼
名稱與 AMI	Name 填 web-1，AMI 選 Amazon Linux 2023
機型	t3.micro
Key pair	選 Proceed without a key pair
Network settings	按 Edit，選 Select existing security group，挑 web-sg
Auto-assign public IP	選 Enable

做完應該看到：五格都填好，還不要按 Launch，先做第 4 步。

實作：第 4 步，掛 Role、貼腳本再開機

同一頁最下面展開 Advanced details；四行是宣告 bash、裝 nginx、寫首頁、啟動並設成開機自動啟動

```
user data

#!/bin/bash
dnf install -y nginx
echo "<h1>hello from `hostname`</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

1 IAM instance profile 選 ec2-ssm-role

2 User data 貼這段，整段複製；它只在第一次開機跑一次

3 按 Launch instance；做完應該看到 Success 跟一個 i- 開頭的 ID

實作：第 5 步，網址要打 http

小步	做什麼	做完應該看到
等開機	在 Instances 列表看 web-1 的 Status check	兩三分鐘後變 2/2 checks passed
拿位址	複製 web-1 的 Public IPv4 address	一串公有 IP
開網頁	瀏覽器打 http://那個IP	hello from 加一串 ip- 開頭的主機名

這台只開 80：網址打成 https 會一直轉圈。

實作：第 6 步，機器活著，服務死了

記住第 3 小步的畫面：高可用單元的兩層健康檢查，就是在解這個問題

1

勾 web-1 → Connect → Session Manager 分頁 → Connect

2

確認 nginx 活著

```
$ systemctl status nginx
```

做完應該看到：綠色 active (running)

3

多玩一步：停掉 nginx，回瀏覽器重新整理

```
$ sudo systemctl stop nginx
```

網頁打不開，Status check 還是 2/2 passed：EC2 的檢查只看機器，不看你的程式

4

看完救回來

```
$ sudo systemctl start nginx
```

實作：第 7 步，清理但留著 Role

小步	做什麼	做完應該看到
先確認	要做進階的人，先做完進階再回來清理	Launch Templates 有 web-template
終止機器	勾 web-1 → Instance state → Terminate	機器列表是 Terminated
刪 SG	等機器變 Terminated 後，刪 web-sg	web-sg 不在列表裡

ec2-ssm-role 留著：高可用單元的實作會用到。

進階：把這台機器存成模板

Launch Template：把開一台機器要填的那一整頁表單存成檔案；Auto Scaling 拿它一直開新機器

1 時機：一定要在第 7 步清理之前

2 勾 web-1 → Actions → Image and templates → Create template from instance

3 名稱填 web-template

驗收 Launch Templates 有 web-template；點進去 Advanced details，User Data 跟 Role 都在。

加分：開著忘了關，一個月付多少

打開 EC2 On-Demand 定價頁，區域選 Tokyo

- 1 查 t3.micro 一小時多少錢
- 2 算「開著忘了關一個月」要付多少
- 3 再查這台的 Spot 價格
- 4 用定價模式那一段的判準，寫一句這台適不適合 Spot

驗收 交一個金額，加一句判斷。

卡住了：先對這四張提示卡

卡在哪	先檢查	怎麼辦
第 5 步一直轉圈	網址是不是打成 https；web-sg 入站是不是 HTTP 80、來源 My IP	改打 http；換過網路就重選一次 My IP
第 5 步是 nginx 預設頁或錯誤頁	User Data 可能沒跑完	Actions → Monitor and troubleshoot → Get system log，找 nginx
第 6 步 Connect 是灰的或 not connected	第 4 步漏選 Role	Actions → Security → Modify IAM role 補上，再 Reboot 等兩三分鐘
第 7 步 web-sg 刪不掉	機器還沒完全 Terminated	等一下再刪

個人作業：就是這個實作，交三張截圖

個人完成；做了進階、加分的再多附一樣

1 必做 70%：網頁截圖 25%、`systemctl status nginx` 畫面 25%、Terminated 畫面 20%

2 進階 20%：附 Launch Template 的截圖

3 加分 10%：附一個月的金額與一句判斷

4 沒清理只扣清理那 20%，並提醒回去看預算告警

驗收 只交必做就及格。

回家作業：替專題的三台機器做決定

六份加上今天的實作，都是個人作業；三台機器是 API 伺服器、跑 agent 的工作機、向量化批次機

定型號

三台各寫吃什麼資源、家族字母、起始大小，加一句為什麼不選隔壁家族。

定付款方式

三台各選定價模式，用 Pricing Calculator 算 On-Demand 跟所選模式的月費。

畫三個 SG

各列入站規則；資料庫那台的來源必須是 SG 參照，再寫 22 埠怎麼處理。

拆安裝清單

至少五項，各標烤進 AMI 或放 User Data；後者寫成可執行的 bash。

改成無金鑰

用 CLI 的 `aws ssm start-session` 進 `t4g.small` 跑 `hostname`，交兩張截圖加指令。

算一筆帳

自架 `g5.xlarge` 一個月對 Bedrock 按 token，寫出每天幾次呼叫才打平。

TAKEAWAY

- 01 型號不用背：會拆名字、會查對照表就夠。
- 02 每一分折扣都是交出去的彈性：先量測，再承諾；GPU 一天坐兩趟就別買車。
- 03 SG 只會說可以；重的先烤進 AMI，擴一台只要幾十秒。
- 04 今天那台機器沒有金鑰、沒有 22 埠，你還是進得去：這是現在的預設做法。

Reference

- **EC2 User Guide : Instance type naming conventions** [名字四段與家族](#)
- **EC2 User Guide : Instance purchasing options 、 Spot Instances** [定價模式](#)
- **Savings Plans User Guide : What is Savings Plans** [承諾金額換折扣](#)
- **EC2 User Guide : Security groups 、 Security group rules** [三個性質與 SG 參照](#)
- **VPC User Guide : Compare security groups and network ACLs** [兩道門](#)
- **EC2 User Guide : Run commands when you launch ; Image Builder User Guide** [User Data](#) 與 [Golden AMI](#)
- **Systems Manager User Guide : Session Manager ; EC2 Instance Connect** [無金鑰連線與實作](#)
- **EC2 User Guide : Accelerated computing instances ; EC2 On-Demand Pricing** [GPU 家族與價格](#)